

Betriebssysteme

Kapitel 4: Prozesse und Nebenläufigkeit

Prof. Dr. Raphael Herding

Inhalte des Kapitels (1)

4. Prozesse und Nebenläufigkeit

- Einordnung & Motivation
- Prozesse
 - Prozessrepräsentation (Prozesskontrollblock)
 - Prozesswechsel (*Context Switch*)
 - Prozesszustände
- Auswahlstrategien (*Scheduling*)
 - *First Come First Served*
 - *Shortest Job First*
 - Prioritäten
 - *Round Robin*
 - *Multilevel-Queue Scheduling*
 - *Multilevel-Feedback-Queue Scheduling*
- ...

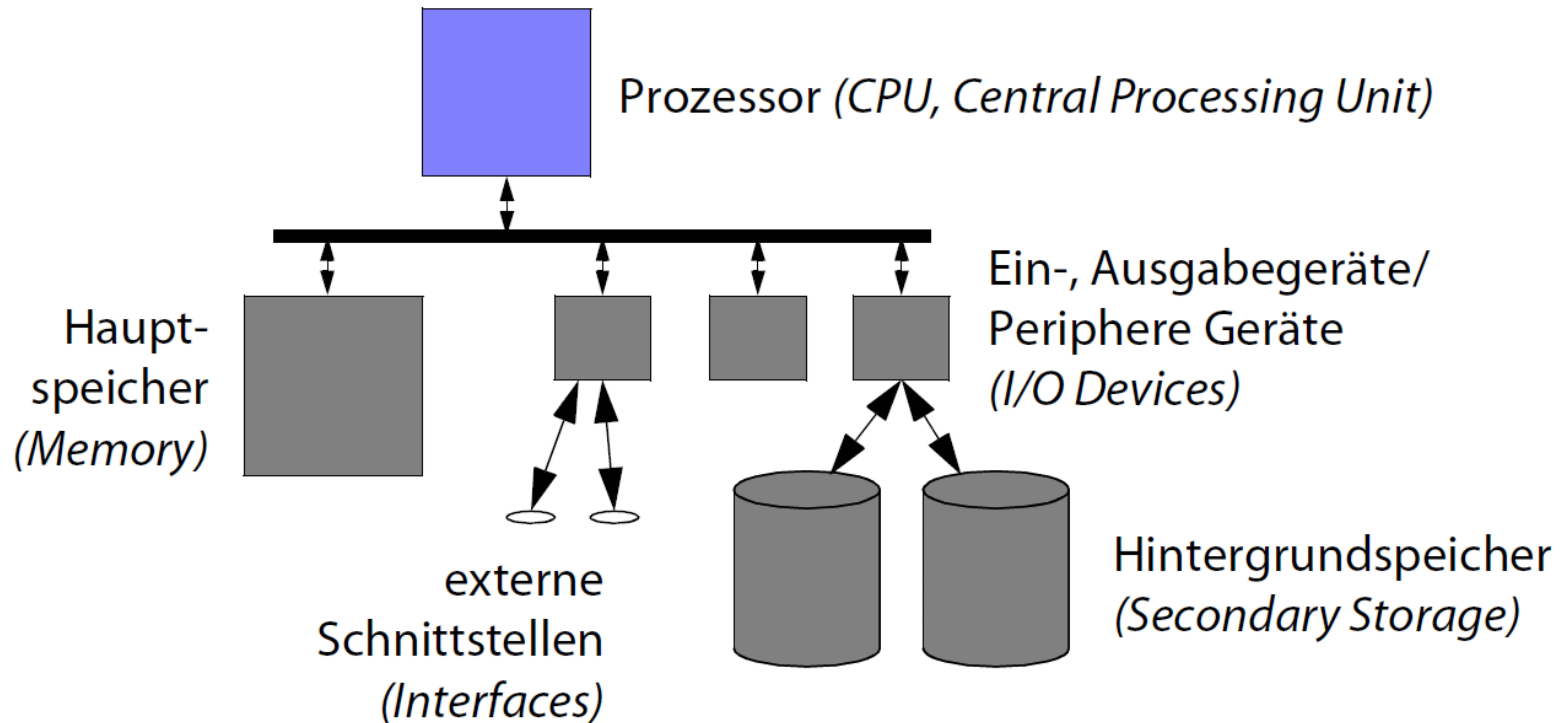
Inhalte des Kapitels (2)

4. Prozesse und Nebenläufigkeit

- ...
- Aktivitätsträger (*Threads*)
 - *User-level Threads*
 - *Kernel-level Threads*
 - *Lightweight Processes*
- Parallelität und Nebenläufigkeit
- Koordinierung
 - Kritische Abschnitte (*Critical Sections*)
 - Wechselseitiger Ausschluss (*Deadlocks*, Algorithmus von Peterson)
 - Spezielle Maschinenbefehle (*Test-and-Set*, *Swap*)
 - Semaphore

Einordnung

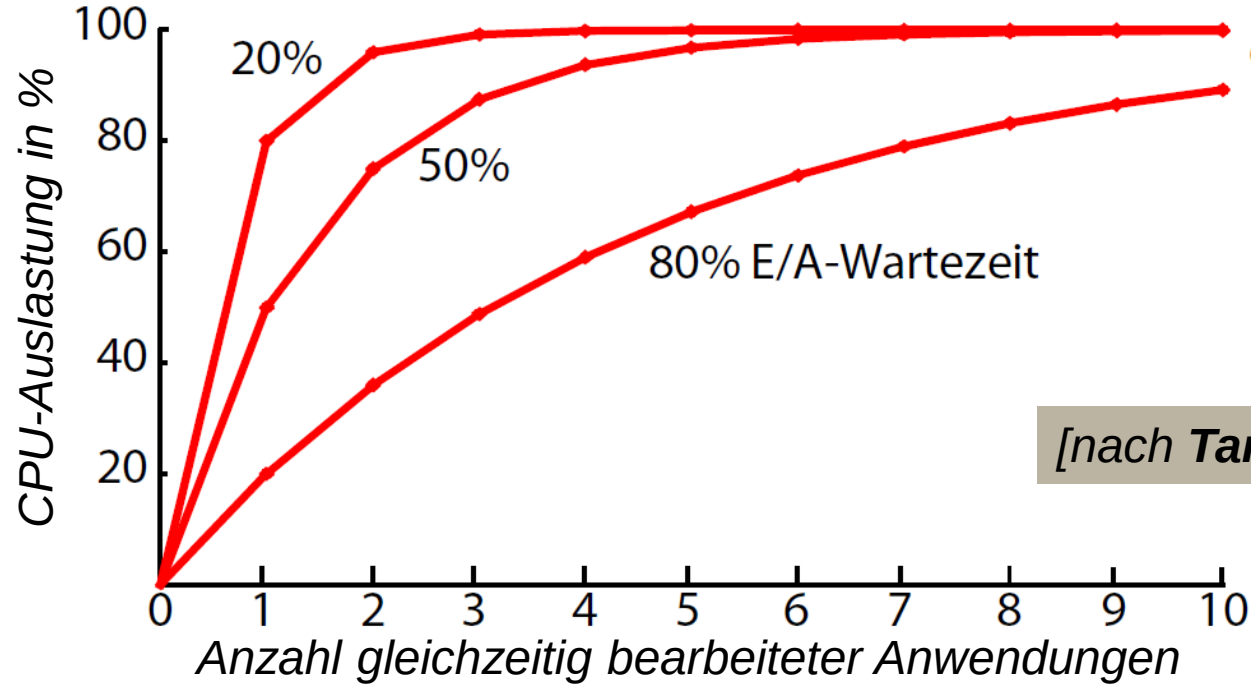
Betroffene physikalische Ressourcen



Motivation (1)

Mehrprozessbetrieb (*Multitasking*)

- Bessere Auslastung der CPU, falls Anwendungen Wartezeiten haben
- Z.B. bei Ein-/Ausgabe



[nach **Tanenbaum**, 1995]

- Einfachere Struktur bei großen Anwendungen

Motivation (2)

Mehrbenutzerbetrieb (*Multi-User*)

- Mehrere Benutzer führen auch mehrere Programme aus
- Ähnlich *Multitasking*-Betrieb
 - Jedoch: Unterscheidung von Benutzern und evtl. Zugriffsberechtigungen

Prozesse

Terminologie

- Programm: Folge von Anweisungen und dazugehörigen Daten
 - Hinterlegt bspw. als Datei auf dem Hintergrundspeicher
- Prozess: Programm, das sich in Ausführung befindet, sowie dessen aktuellen Daten
 - *Beachte*: Ein Programm kann sich mehrfach in Ausführung befinden, d.h. es kann mehrere Prozesse zu einem Programm geben

Prozess stellt Ausführungsumgebung bereit

- Adressraum (Schutzumgebung für den Speicher eines Prozesses)
- Kontext für Ressourcenanforderungen (Speicher, Dateien, etc.)
- Prozess als virtueller Prozessor
 - Anweisungen des Prozesses werden abgearbeitet

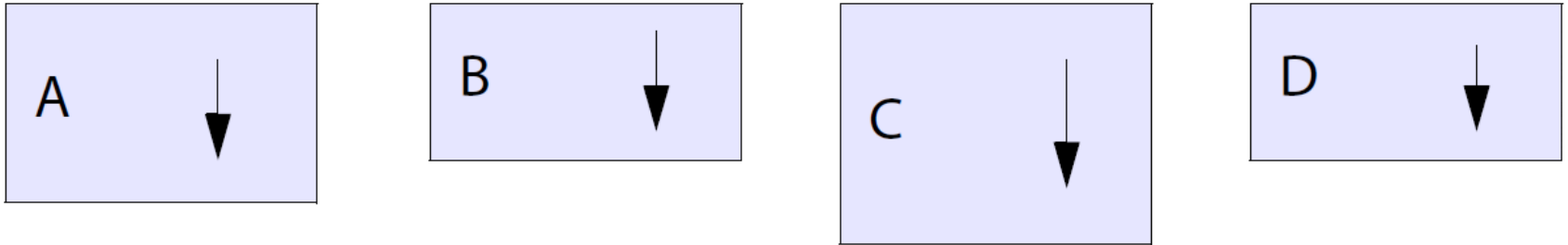
Repräsentation / Datenstruktur pro Prozess

Prozesskontrollblock (*Process Control Block, PCB*)

- Enthält alle nötigen Daten für einen Prozess
- Beispielsweise in UNIX/Linux:
 - Prozessnummer (*process identifier, PID*)
 - Verbrauchte Rechenzeit
 - Erzeugungszeitpunkt
 - Kontext (Register etc., insbes. PC)
 - Speicherabbildung (Information zur Schutzumgebung)
 - Eigentümer (*user ID UID, group ID GID*)
 - Wurzelverzeichnis, aktuelles Verzeichnis
 - Offene Dateien
 - ...

Prozesswechsel (1)

Konzeptionelles Modell



Vier Prozesse mit eigenständigen Programmzählern

Umschaltung zwischen den Prozessen

- Ein Prozessor kann zu jeder Zeit immer nur einen Prozess abarbeiten
- Umschaltung zwischen den Prozessen
 - Prozesse erleben Fortschritt ihrer Programmzähler und ihrer ausgeführten Anweisungen
- Transparentes Umschalten
 - Prozess bemerkt andere Prozesse (zunächst) nicht

Prozesswechsel (2)

Prozessumschaltung (Kontextwechsel, *Context Switch*)

- Sichern der Register des laufenden Prozesses, inkl. Programmzähler (Kontext)
- Auswahl des neuen Prozesses
- Ablaufumgebung des neuen Prozesses herstellen (z.B. Speicherabbildung, etc.)
- Gesicherte Register laden, und
- Prozessor aufsetzen, d.h. Programmzähler laden

Prozesswechsel (3)

Prozesswechsel unter Kontrolle der Prozesse

- Gerade laufender Prozess bestimmt Zeitpunkt des Kontextwechsels
 - Konzept der Co-Routine
- Kein transparentes Umschalten
- Keine Fairness

Wunsch nach fairer Zuteilung der Ressource „Prozessor“

Prozesswechsel (4)

Prozesswechsel unter der Kontrolle des Betriebssystems

- Mögliche Eingriffspunkte:
 - Systemaufrufe
 - Unterbrechungen
- Wechsel nach/in Systemaufrufen
 - Warten auf Ereignisse (z.B. Zeitpunkt, Nachricht, Lesen Plattenblock)
 - Terminieren des Prozesses
- Wechsel nach *Interrupts*
 - Ablauf einer Zeitscheibe
 - Bevorzugter Prozess wurde laufbereit

Auswahlstrategie zur Wahl des nächsten Prozesses

- *Scheduler*-Komponente des Betriebssystems

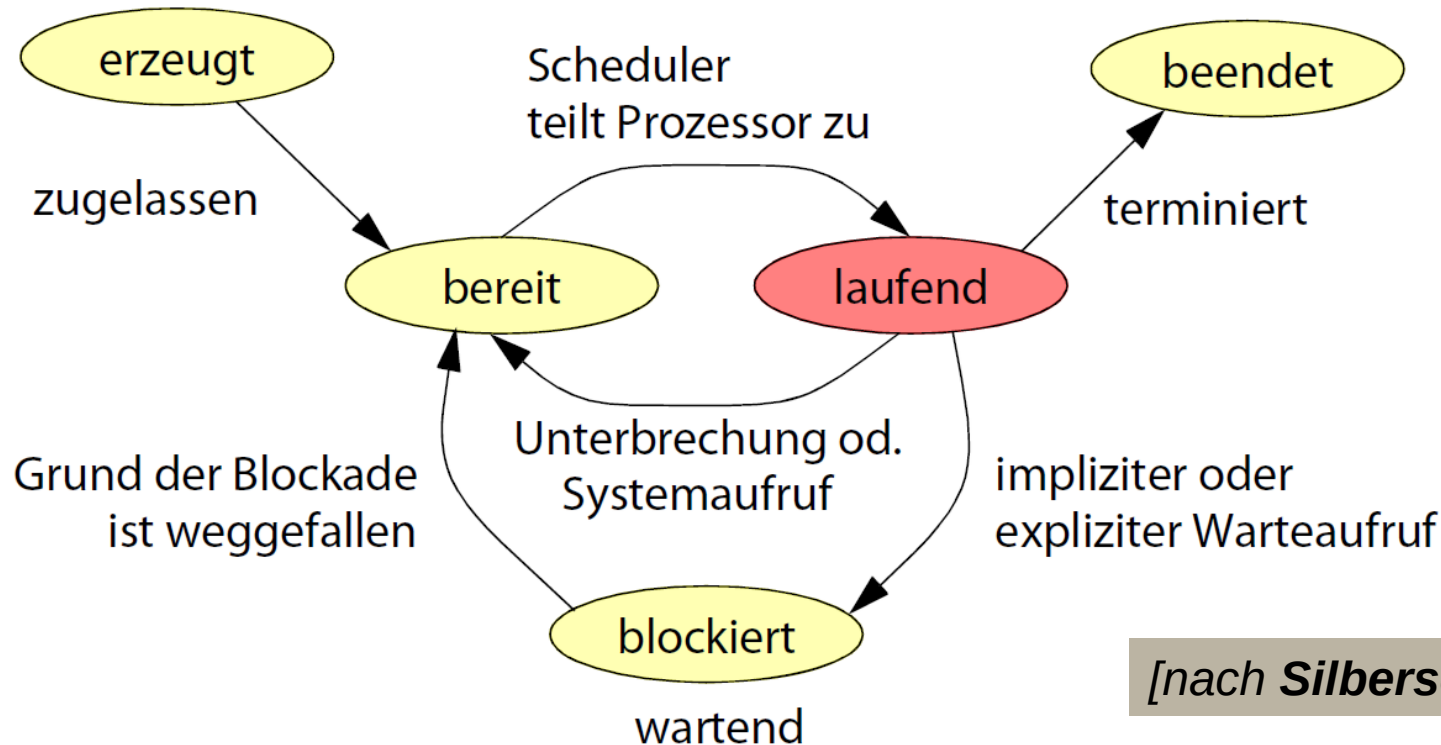
Prozesszustände (1)

Ein Prozess befindet sich in einem der folgenden Zustände

- Erzeugt (*New*)
Prozess wurde erzeugt, besitzt aber noch nicht alle nötigen Ressourcen
- Bereit (*Ready*)
Prozess besitzt alle nötigen Ressourcen und ist bereit zum Laufen
- Laufend (*Running*)
Prozess wird auf einem realen Prozessor ausgeführt
- Blockiert (*Blocked / Waiting*)
Prozess wartet auf ein Ereignis (z.B. Fertigstellung einer I/O-Operation, Zuteilung einer Ressource, Empfang einer Nachricht); zum Warten wird er blockiert
- Beendet (*Terminated*)
Prozess ist beendet, einige Ressourcen sind aber noch nicht freigegeben oder Prozess muss aus anderen Gründen im System verbleiben

Prozesszustände (2)

Zustandsdiagramm



[nach **Silberschatz**, 1994]

- Scheduler ist der Teil des Betriebssystems, der die Zuteilung des realen Prozessors vornimmt

Roter Faden

4. Prozesse und Nebenläufigkeit

- Einordnung & Motivation
- Prozesse
 - Prozessrepräsentation (Prozesskontrollblock)
 - Prozesswechsel (*Context Switch*)
 - Prozesszustände
- Auswahlstrategien (*Scheduling*)
- Aktivitätsträger (*Threads*)
- Parallelität und Nebenläufigkeit
- Koordinierung

Strategien zur Auswahl des nächsten Prozesses

Scheduling Strategies

- Mögliche Stellen zum Treffen von *Scheduling*-Entscheidungen
 1. Prozess wechselt von „**laufend**“ nach „**blockiert**“ (z.B. bei I/O)
 2. Prozess wechselt von „**laufend**“ nach „**bereit**“ (z.B. bei einer Unterbrechung des Prozessors)
 3. Prozess wechselt von „**blockiert**“ nach „**bereit**“
 4. Prozess terminiert
- Bei 1. und 4. *muss* Neuauswahl erfolgen
- Bei 2. und 3. *kann* Neuauswahl erfolgen

verdrängend = *präemptiv*

verdrängend

nicht verdrängend

Kriterien für *Scheduling*-Strategien

- CPU-Auslastung
 - Möglichst zu 100% ausgelastete CPU
- Durchsatz
 - Möglichst hohe Anzahl bearbeiteter Prozesse pro Zeiteinheit
- Verweilzeit
 - Möglichst geringe Gesamtzeit des Prozesses in der Rechenanlage
- Wartezeit
 - Möglichst kurze Gesamtzeit, in der der Prozess im Zustand „bereit“ ist
- Antwortzeit
 - Möglichst kurze Reaktionszeit des Prozesses in interaktivem Betrieb

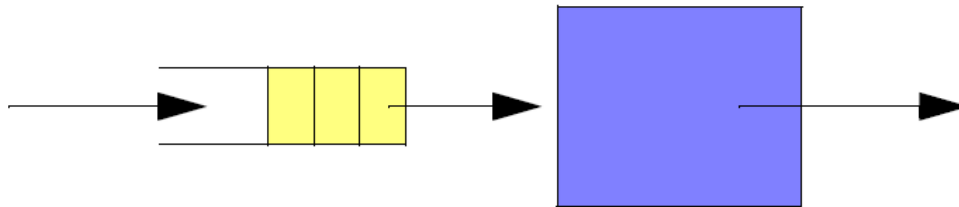
First Come, First Served (1)

Der erste Prozess wird zuerst bearbeitet (FCFS)

- „Wer zuerst kommt...“
- Nicht-präemptiv

Warteschlange zum Zustand „bereit“

- Prozesse werden hinten eingereiht
- Prozesse werden vorne entnommen



Bewertung

- Fair (?)
- Wartezeiten nicht minimal
- Nicht für *Time-Sharing* Betrieb geeignet

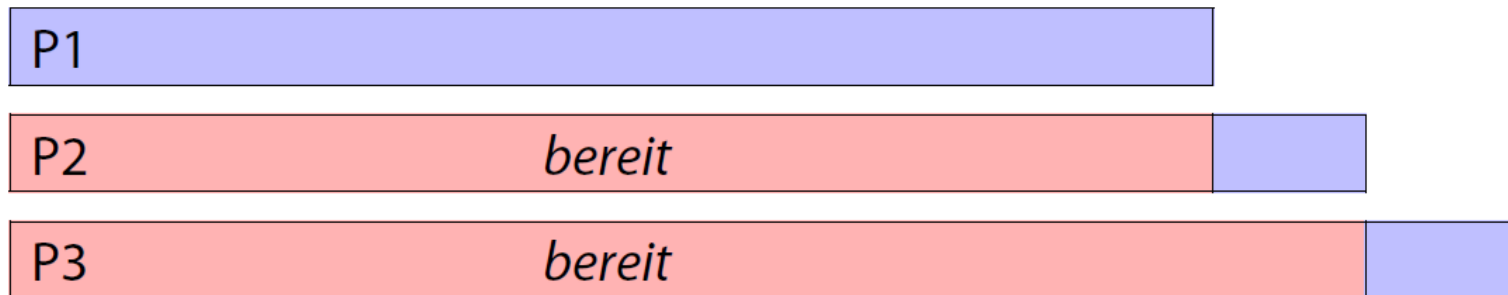
First Come, First Served (2)

Beispiel zur Betrachtung der Wartezeiten

Prozess 1: 24
Prozess 2: 3
Prozess 3: 3

} Zeiteinheiten

– Reihenfolge: P1, P2, P3



Mittlere Wartezeit: $(0 + 24 + 27) / 3 = 17$

■ laufend
■ bereit
■ blockiert

First Come, First Served (3)

Beispiel zur Betrachtung der Wartezeiten

Prozess 1: 24
Prozess 2: 3
Prozess 3: 3

} Zeiteinheiten

– Reihenfolge: P2, P3, P1



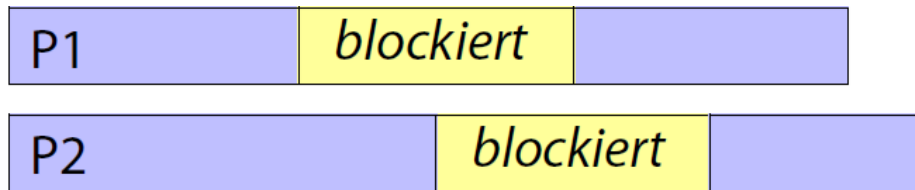
Mittlere Wartezeit: $(6 + 0 + 3) / 3 = 3$

laufend
bereit
blockiert

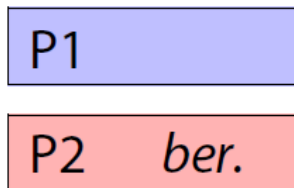
First Come, First Served (4)

Behandlung deblockierter Prozesse

- Einreihung an das Ende der Warteschlange
- Beispiel
 - Laufzeitanforderung

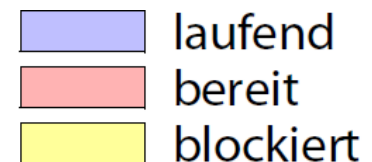


- FCFS-Ablauf auf einem Prozessor (initial: P1 vor P2)



Warte- P1
schlange: P2

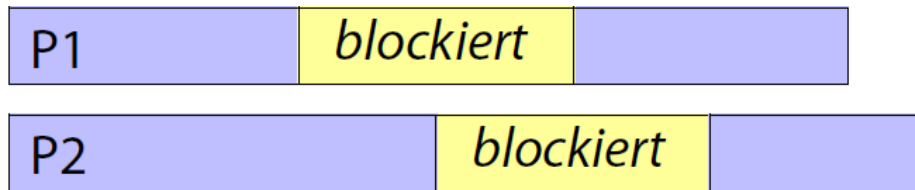
P2



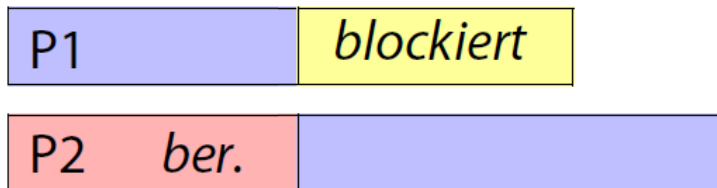
First Come, First Served (5)

Behandlung deblockierter Prozesse

- Einreihung an das Ende der Warteschlange
- Beispiel
 - Laufzeitanforderung



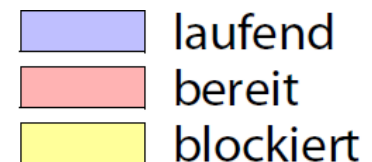
- FCFS-Ablauf auf einem Prozessor (initial: P1 vor P2)



Warte- P1
schlange: P2

P2

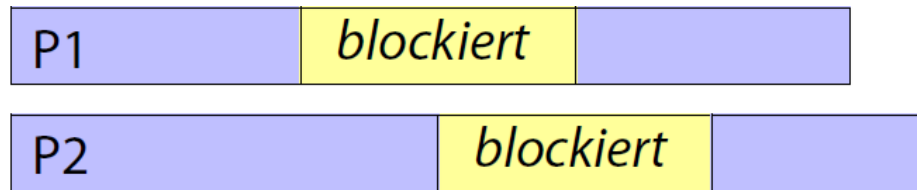
P1



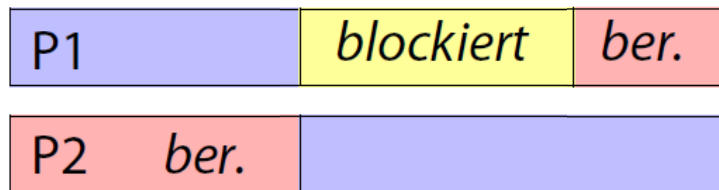
First Come, First Served (6)

Behandlung deblockierter Prozesse

- Einreihung an das Ende der Warteschlange
- Beispiel
 - Laufzeitanforderung



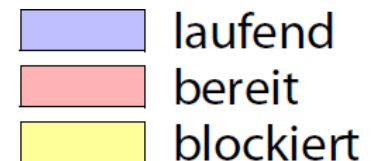
- FCFS-Ablauf auf einem Prozessor (initial: P1 vor P2)



Warte- P1
schlange: P2

P2

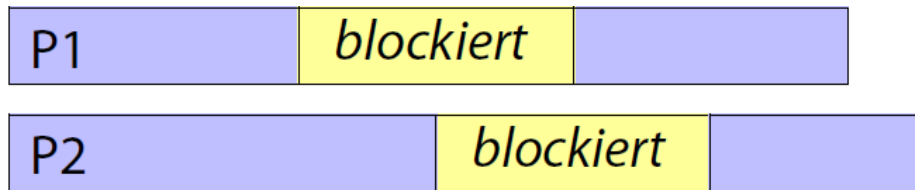
P1 P1



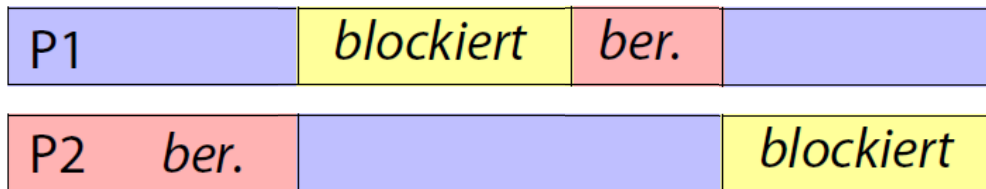
First Come, First Served (7)

Behandlung deblockierter Prozesse

- Einreihung an das Ende der Warteschlange
- Beispiel
 - Laufzeitanforderung

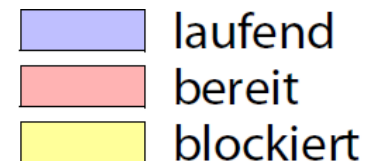


- FCFS-Ablauf auf einem Prozessor (initial: P1 vor P2)



Warte- P1
schlange: P2

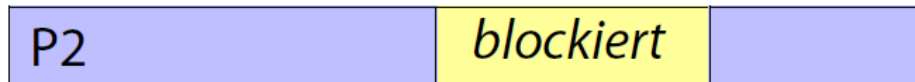
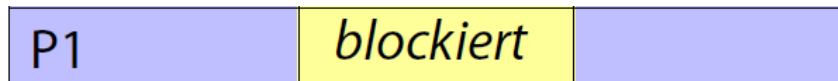
P2 P1 P1 P2



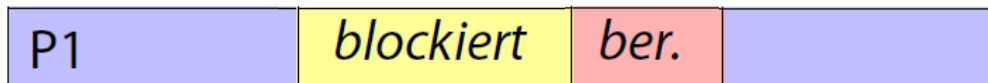
First Come, First Served (8)

Behandlung deblockierter Prozesse

- Einreihung an das Ende der Warteschlange
- Beispiel
 - Laufzeitanforderung

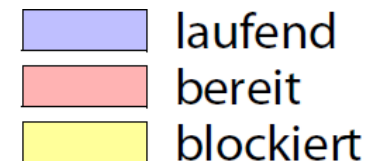


- FCFS-Ablauf auf einem Prozessor (initial: P1 vor P2)



Warte- P1
schlange: P2

P2 P1 P1 P2



Shortest Job First (1)

Kürzester Job wird ausgewählt (SJF)

- Länge bezieht sich auf die nächste Rechenphase bis zur nächsten Warteoperation (z.B. I/O)

„Bereit“-Warteschlange wird nach Länge der nächsten Rechenphase sortiert

- Vorhersage der Länge durch Protokollieren der Länge bisheriger Rechenphasen (Mittelwert, exponentielle Glättung)
- ... Protokollierung der Länge der vorherigen Rechenphase

SJF optimiert die mittlere Wartezeit

- Da Länge der Rechenphase in der Regel nicht genau vorhersagbar, nicht ganz optimal

Varianten: präemptiv (PSJF) und nicht-präemptiv

Prioritäten (1)

Prozess mit höchster Priorität wird ausgewählt

(HPF, *Highest Priority First*)

- Dynamisch – statisch
(z.B. SJF: dynamische Vergabe von Prioritäten gemäß Länge der nächsten Rechenphase)
(z.B. statische Prioritäten in Echtzeitsystemen; Vorhersagbarkeit von Reaktionszeiten)
- Verdrängend – nicht-verdrängend

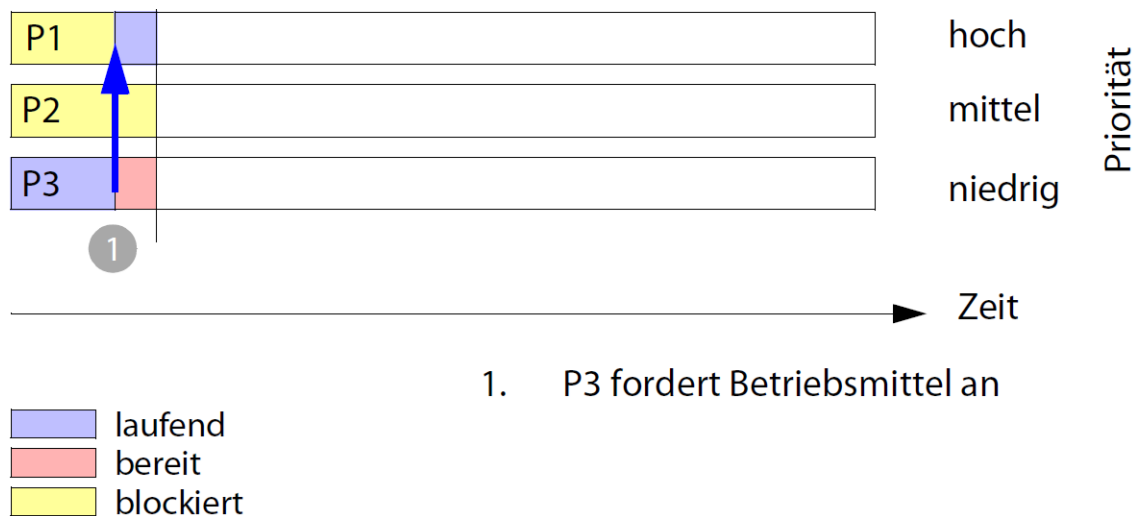
Probleme

- Prioritätsumkehr (*Priority Inversion*)
- Aushungerung (*Starvation*)

Prioritäten (2)

Prioritätsumkehr

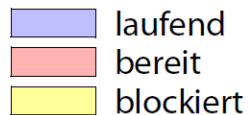
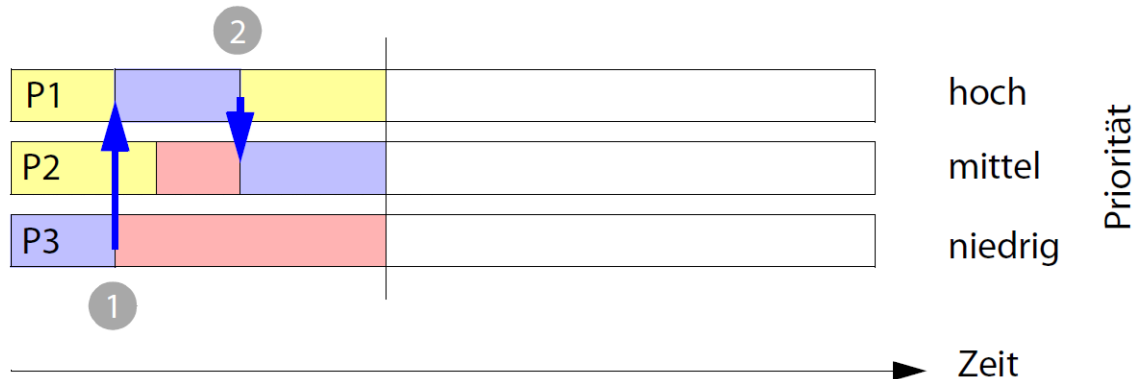
- Hochpriorer Prozess wartet auf eine Ressource, die ein niedrigpriorer Prozess besitzt.
Dieser wiederum wird durch einen mittelprioren Prozess verdrängt und kann daher die Ressource nicht freigeben
- ☞ Hochpriorer Prozess muss auf andere mit niedrigerer Priorität warten!



Prioritäten (3)

Prioritätsumkehr

- Hochprioriter Prozess wartet auf eine Ressource, die ein niedrigprioriter Prozess besitzt.
- Dieser wiederum wird durch einen mittelprioren Prozess verdrängt und kann daher die Ressource nicht freigeben
- ☞ Hochprioriter Prozess muss auf andere mit niedrigerer Priorität warten!



1. P3 fordert Betriebsmittel an
2. P1 wartet auf das gleiche Betriebsmittel

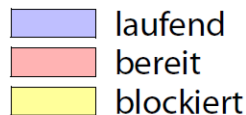
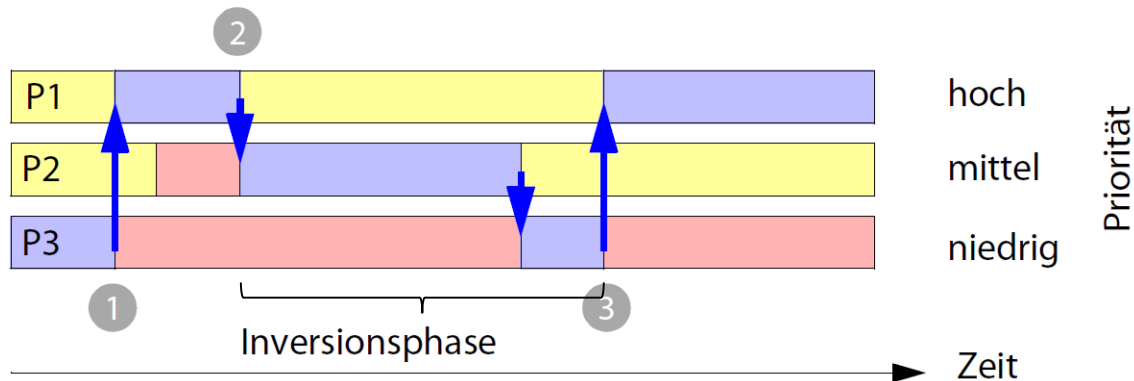
Prioritäten (4)

Prioritätsumkehr

- Hochpriorer Prozess wartet auf eine Ressource, die ein niedrigpriorer Prozess besitzt.

Dieser wiederum wird durch einen mittelprioren Prozess verdrängt und kann daher die Ressource nicht freigeben

☞ Hochpriorer Prozess muss auf andere mit niedrigerer Priorität warten!



1. P3 fordert Betriebsmittel an
2. P1 wartet auf das gleiche Betriebsmittel
3. P3 gibt Betriebsmittel frei

Multilevel-Queue Scheduling (1)

Verschiedene *Scheduling*-Klassen

- Z.B. Hintergrundprozesse (*Batch*) und Vordergrundprozesse (interaktive Prozesse)
- Jede Klasse besitzt ihre eigenen Warteschlangen (*Queues*) und verwaltet diese nach einer eigenen *Scheduling*-Strategie
- Zwischen den Klassen gibt es ebenfalls eine *Scheduling*-Strategie
Z.B. feste Prioritäten (Vordergrundprozesse immer vor Hintergrundprozessen)
- Prozessauswahl
 1. *Scheduling*-Klasse auswählen
 2. In ausgewählter Klasse Prozess auswählen

Multilevel-Queue Scheduling (2)

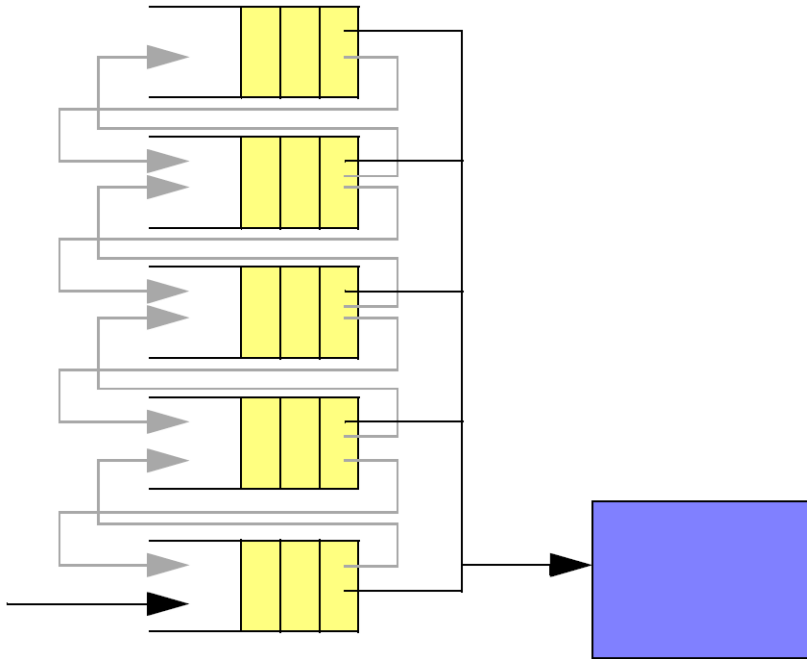
Beispiel: Linux

- *Scheduling*-Klassen
 - Systemprozesse
 - *Real-Time* Prozesse
 - *Time-Sharing* Prozesse
 - Interaktive Prozesse
- *Scheduling* zwischen den Klassen mit fester Priorität
(z.B. *Real-Time* Prozesse vor *Time-Sharing* Prozessen)

Multilevel-Queue Scheduling (2)

Beispiel

- 5 *Scheduling*-Klassen mit eigener *Ready Queue* und Strategie
- Prozesse können von einer zur anderen *Queue* transferiert werden



Roter Faden

4. Prozesse und Nebenläufigkeit

- Einordnung & Motivation
- Prozesse
- Auswahlstrategien (*Scheduling*)
 - *First Come First Served*
 - *Shortest Job First*
 - Prioritäten
 - *Multilevel-Queue Scheduling*
- Aktivitätsträger (*Threads*)
- Parallelität und Nebenläufigkeit
- Koordinierung

Zusammenfassung (Prozesse)

Mehrere Prozesse zur Strukturierung von Problemlösungen

- Aufgaben eines Prozesses leichter modellierbar, wenn in mehrere kooperierende Prozesse unterteilt
 - z.B. Anwendungen mit mehreren Fenstern (ein Prozess pro Fenster)
 - z.B. Anwendungen mit vielen gleichzeitigen Aufgaben (Webbrowser)
- Multiprozessor-Systeme werden erst mit mehreren parallel laufenden Prozessen ausgenutzt
 - z.B. wissenschaftliches Hochleistungsrechnen (Klima-Simulation, Strömungssimulation etc.)
- *Client-Server* Anwendungen unter UNIX: pro Anfrage wird ein neuer Prozess gestartet
 - z.B. Webserver

Prozesse mit gemeinsamem Speicher

Kommunikation zwischen kooperierenden Prozessen

- Üblicherweise über gemeinsame Nutzung von Speicherbereichen durch mehrere Prozesse

Vorteile

- Sehr einfach zu programmieren
- In Multiprozessor-Systemen sind echt parallele Abläufe möglich

Nachteile

- Viele Ressourcen sind zur Verwaltung eines Prozesses notwendig
 - *File*-Deskriptoren
 - Speicherabbildung
 - Prozesskontrollblock
- Prozesswechsel sind aufwändig

Aktivitätsträger (1)

Alternative:

Aktivitätsträger (*Threads*) oder

- Eine Gruppe von *Threads* nutzt gemeinsam eine Menge von Ressourcen
 - Instruktionen
 - Datenbereiche
 - *Files, Sockets* etc.
- Jeder *Thread* repräsentiert eine eigene Aktivität
 - Eigener Programmzähler
 - Eigener Registersatz
 - Eigener *Stack*

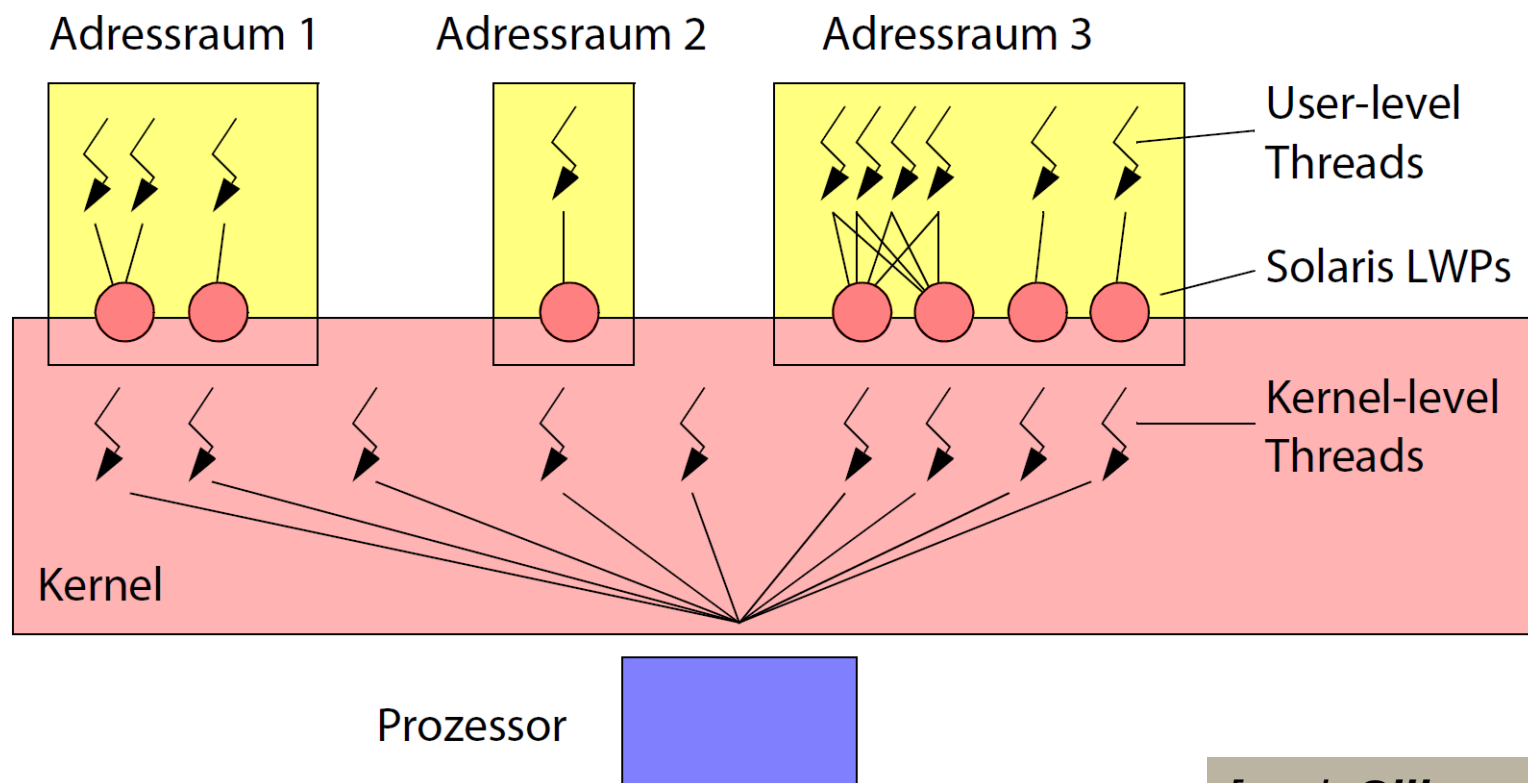
Aktivitätsträger (2)

- Umschalten zwischen zwei *Threads* einer Gruppe ist erheblich billiger als ein normaler Kontextwechsel
 - Es müssen nur die Register und der Programmzähler gewechselt werden (entspricht dem Aufwand für einen Funktionsaufruf)
 - Speicherabbildung muss nicht gewechselt werden
 - Alle Systemressourcen bleiben verfügbar

Linux/UNIX-Prozesse

- Sind Adressräume mit zunächst einem *Thread*
- Weitere *Threads* können dynamisch zur Laufzeit erzeugt werden

Beispiel: *Threads* (Linux)



[nach **Silberschatz**, 1994]

[http://en.wikipedia.org/wiki/Light-weight_process]

Roter Faden

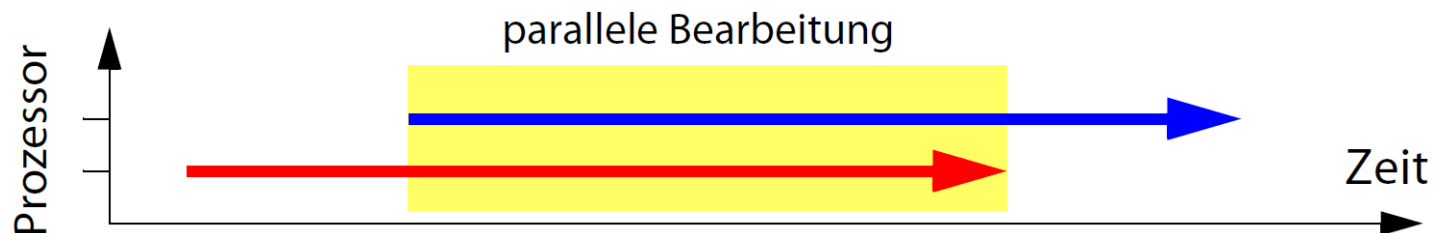
4. Prozesse und Nebenläufigkeit

- Einordnung & Motivation
- Prozesse
- Auswahlstrategien (*Scheduling*)
- Aktivitätsträger (*Threads*)
- Parallelität und Nebenläufigkeit
- Koordinierung

Parallelität und Nebenläufigkeit (1)

Parallelität bei mehreren Prozessen oder *Threads*

- Die Anweisungen zweier Prozesse werden parallel bearbeitet, wenn die Anweisungen unabhängig voneinander zur gleichen Zeit ausgeführt werden
- Parallele Bearbeitung nur auf Multiprozessoren
 - Zwei oder mehr Prozessoren können parallel Anweisungen bearbeiten

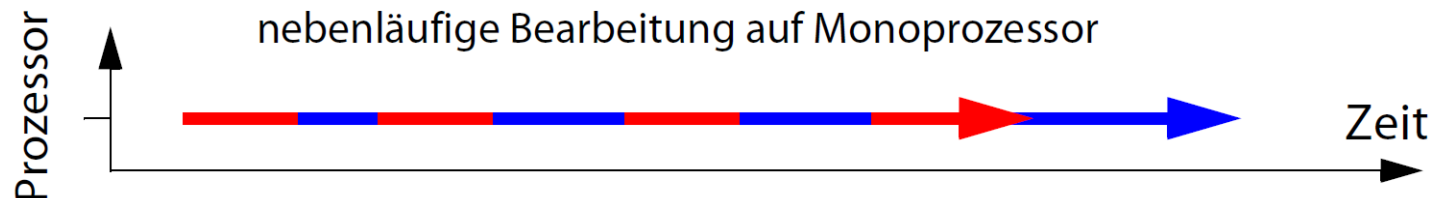


- Monoprozessor
 - Keine parallele Abarbeitung von Befehlen möglich

Parallelität und Nebenläufigkeit (2)

Nebenläufigkeit

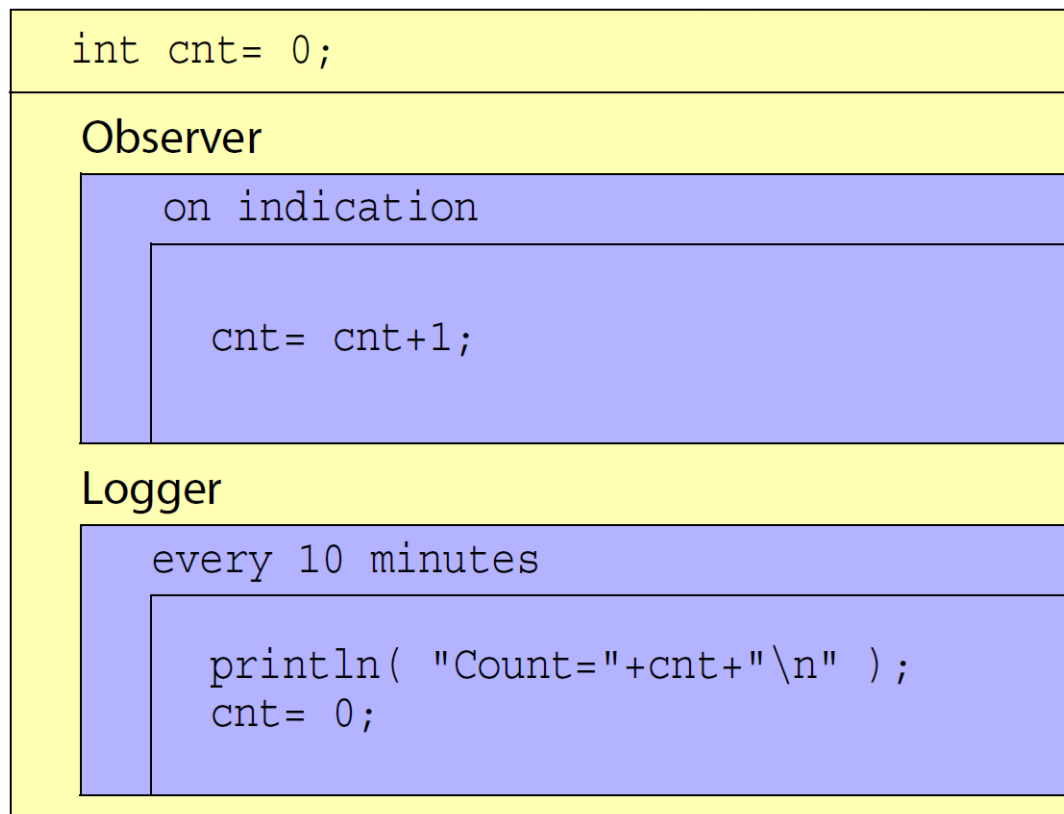
- Zwei Prozesse heißen nebenläufig, wenn ihre Anweisungen unabhängig voneinander abgearbeitet werden. Dabei spielt es keine Rolle, ob die Anweisungen zeitlich durchmischt oder auch echt gleichzeitig bearbeitet werden.
- Parallele Bearbeitung ist nebenläufig
- Nebenläufige Bearbeitung auch auf Monoprozessoren möglich
 - Zeitliche Durchmischung der Befehle mehrerer Prozesse / *Threads*
 - *Scheduling*-Strategie oft unabhängig von den bearbeiteten Befehlen



Koordinierung (1)

Beispiel: zwei Prozesse – Beobachter und Protokollierer

- Per Schleife werden Fahrzeuge gezählt. Alle 10 Minuten druckt der Protokollierer die im letzten Zeitraum vorbeigekommene Anzahl aus.



Koordinierung (2)

Effekte

- Fahrzeuge gehen „verloren“
- Fahrzeuge werden doppelt gezählt

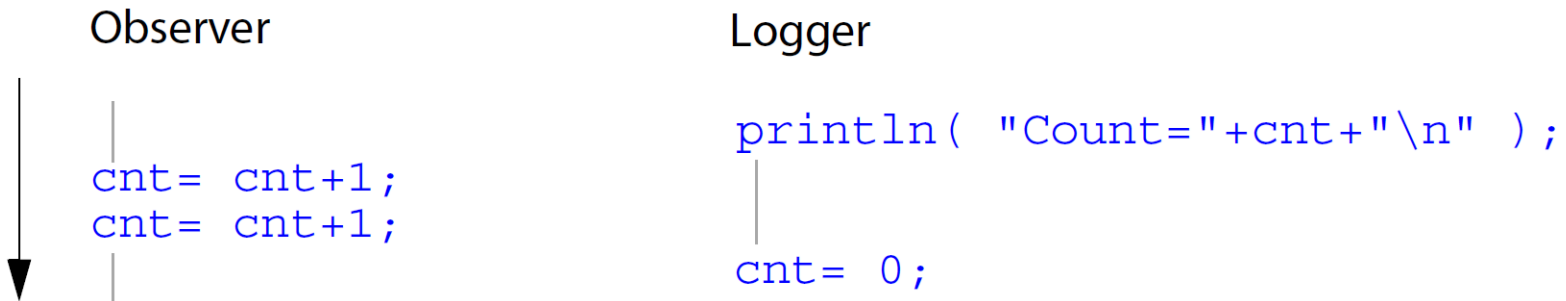
Ursachen

- Befehle einer Programmiersprache werden nicht unteilbar (atomar) abgearbeitet, da sie auf mehrere Maschinenbefehle abgebildet werden
- Keinesfalls werden mehrere Anweisungen zusammen atomar abgearbeitet
- Prozesswechsel innerhalb einer Anweisung oder zwischen zwei zusammengehörigen Anweisungen können zu Inkonsistenzen führen

Koordinierung (3)

Fahrzeuge gehen „verloren“

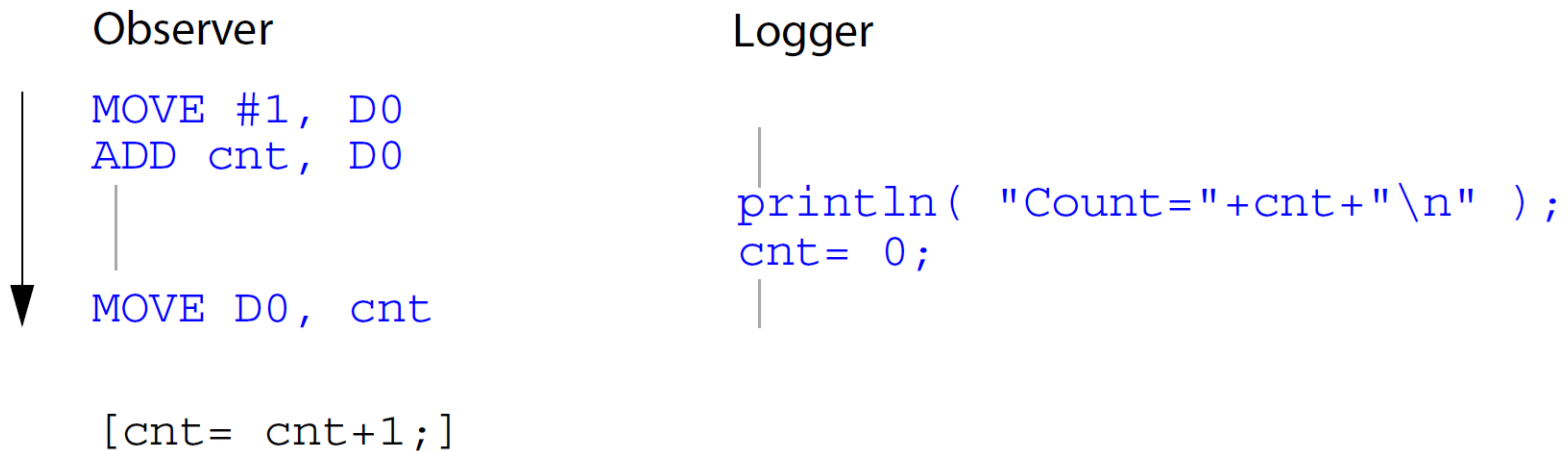
- Nach dem Drucken wird der Protokollierer unterbrochen.
Beobachter zählt weitere Fahrzeuge.
Zähler wird danach ohne Beachtung vom *Logger* auf Null gesetzt.



Koordinierung (4)

Fahrzeuge werden doppelt gezählt

- Beobachter will Zähler erhöhen und holt sich diesen dazu in ein Register. Beobachter wird unterbrochen, *Logger* setzt Zähler auf Null. Beobachter erhöht Registerwert und schreibt diesen zurück. Dieser Wert wird erneut vom Protokollierer registriert.



Koordinierung (5)

Problem gekoppelt an gemeinsame Nutzung von Daten / Ressourcen

Lösung: Kritische Abschnitte (*Critical Sections*)

- Nur ein Prozess/*Thread* soll Zugang zu Daten/Ressourcen haben (wechselseitiger Ausschluss, *Mutual Exclusion*, *Mutex*)
- Kritische Abschnitte erscheinen als zeitlich unteilbar

Wie kann der wechselseitige Ausschluss in kritischen Abschnitten erzielt werden?

- ☞ Vorkehrungen, dass nicht mehrere Prozesse gleichzeitig im kritischen Abschnitt sind

Koordinierung (6)

Koordinierung allgemein

- Einschränkung der zeitlichen Durchmischung von Befehlsfolgen in nebenläufigen Prozessen/*Threads*
- Letztlich Einschränkung der Nebenläufigkeit
 - ☞ Gezielte Aufgabe der Unabhängigkeit bei der Befehlsausführung

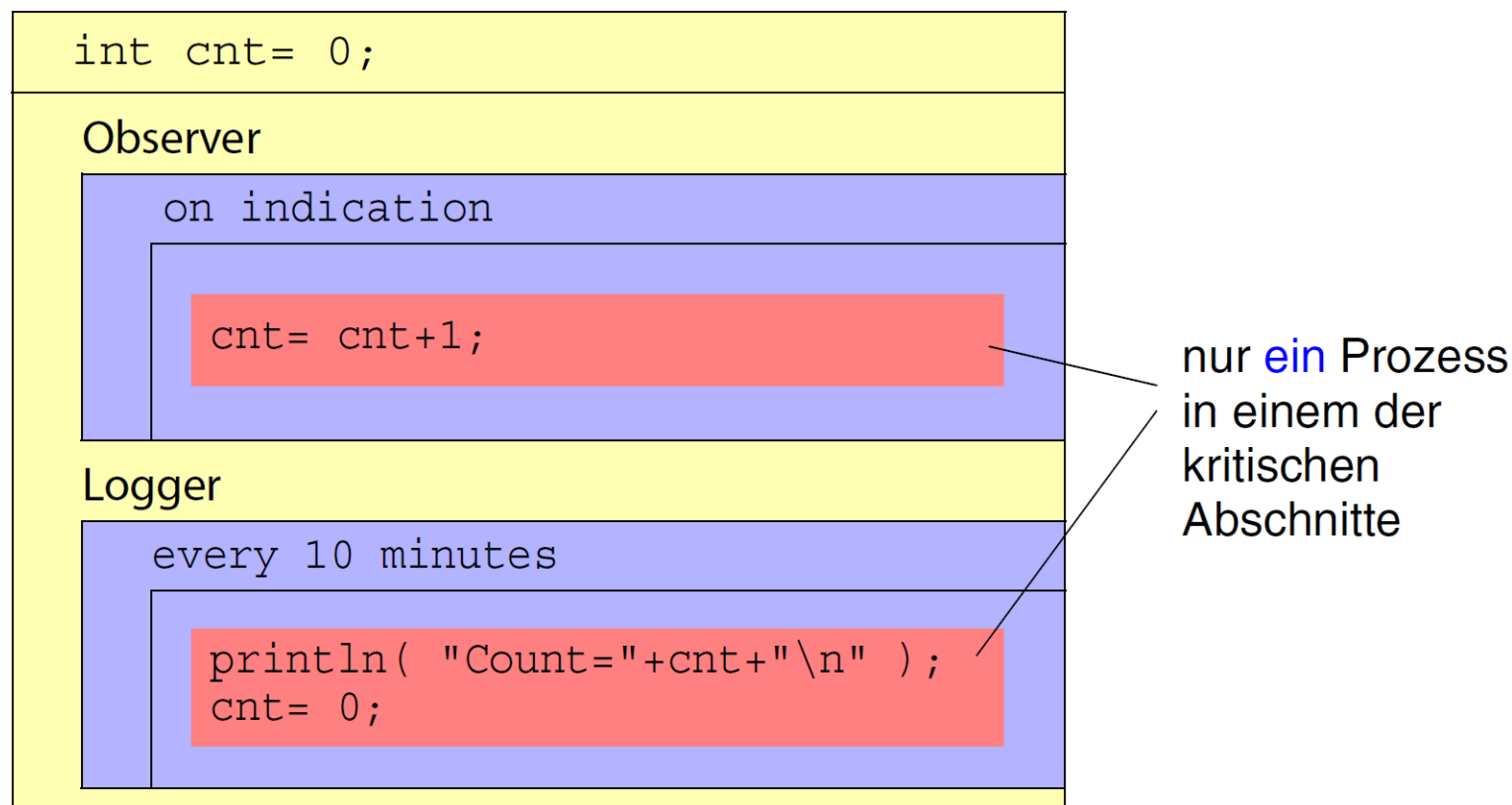
Hinweis

- ☞ Im Folgenden wird immer von Prozessen die Rede sein.
Koordinierung kann/muss selbstverständlich auch zwischen *Threads* stattfinden.

Wechselseitiger Ausschluss (1)

Lösung des Beispiels

- Zwei zusammengehörige kritische Abschnitte



Wechselseitiger Ausschluss (2)

Zwei Prozesse wollen regelmäßig kritischen Abschnitt betreten

- Annahme: Maschinenbefehle sind unteilbar (atomar)

1. Versuch

```
int turn= 0;
```

Prozess 0

```
while( 1 ) {  
    while( turn == 1 );  
    ...  
    /* critical sec. */  
    ...  
    turn= 1;  
    ... /* uncritical */  
}
```

Prozess 1

```
while( 1 ) {  
    while( turn == 0 );  
    ...  
    /* critical sec. */  
    ...  
    turn= 0;  
    ... /* uncritical */  
}
```

Wechselseitiger Ausschluss (3)

Probleme der Lösung

- Nur alternierendes Betreten des kritischen Abschnitts durch P_0 und P_1 möglich
- Implementierung ist unvollständig
- Aktives Warten

Ersetzen von `turn` durch zwei Variablen `ready0` und `ready1`

- `ready0` zeigt an, dass Prozess 0 bereit für den kritischen Abschnitt ist
- `ready1` zeigt an, dass Prozess 1 bereit für den kritischen Abschnitt ist

Wechselseitiger Ausschluss (4)

2. Versuch

```
boolean ready0= false;  
boolean ready1= false;
```

Prozess 0

```
while( 1 ) {  
    ready0= true;  
    while( ready1 );  
  
    ... /* critical sec. */  
  
    ready0= false;  
  
    ... /* uncritical */  
}
```

Prozess 1

```
while( 1 ) {  
    ready1= true;  
    while( ready0 );  
  
    ... /* critical sec. */  
  
    ready1= false;  
  
    ... /* uncritical */  
}
```

Wechselseitiger Ausschluss (5)

Wechselseitiger Ausschluss wird erreicht

- Leicht nachweisbar durch Zustände von **ready0** und **ready1**

Probleme der Lösung

- Aktives Warten
- Verklemmung (*Deadlock*) möglich

Wechselseitiger Ausschluss (6)

Betrachtung der nebenläufigen Abfolgen

P_0

```

ready0= true;
while( ready1 ); +
<critical>      +
ready0= false;
<noncritical>   +
ready0= true;
...
  
```

P_1

```

ready1= true;
while( ready0 ); +
<critical>      +
ready1= false;
<noncritical>   +
ready1= true;
...
  
```

+ = mehrfach, mind. einmal
 * = mehrfach oder gar nicht

- Durchspielen aller möglichen Durchmischungen

Wechselseitiger Ausschluss (7)

Harmlose Durchmischung

*Ausgeführte
Anweisungen*

P_0

P_1

```

ready0= true;
while( ready1 ); +
<critical>      +
ready0= false;
<noncritical>   +
ready0= true;
...

```

```

ready1= true;
while( ready0 ); +
<critical>      +
ready1= false;
<noncritical>   +
ready1= true;
...

```

Wechselseitiger Ausschluss (7)

Harmlose Durchmischung

*Ausgeführte
Anweisungen*

P_0

P_1

`ready0= true;`

`while( ready1 );` +

`<critical>` +

`ready0= false;`

`<noncritical>` +

`ready0= true;`

`...`

`ready1= true;`

`while( ready0 );` +

`<critical>` +

`ready1= false;`

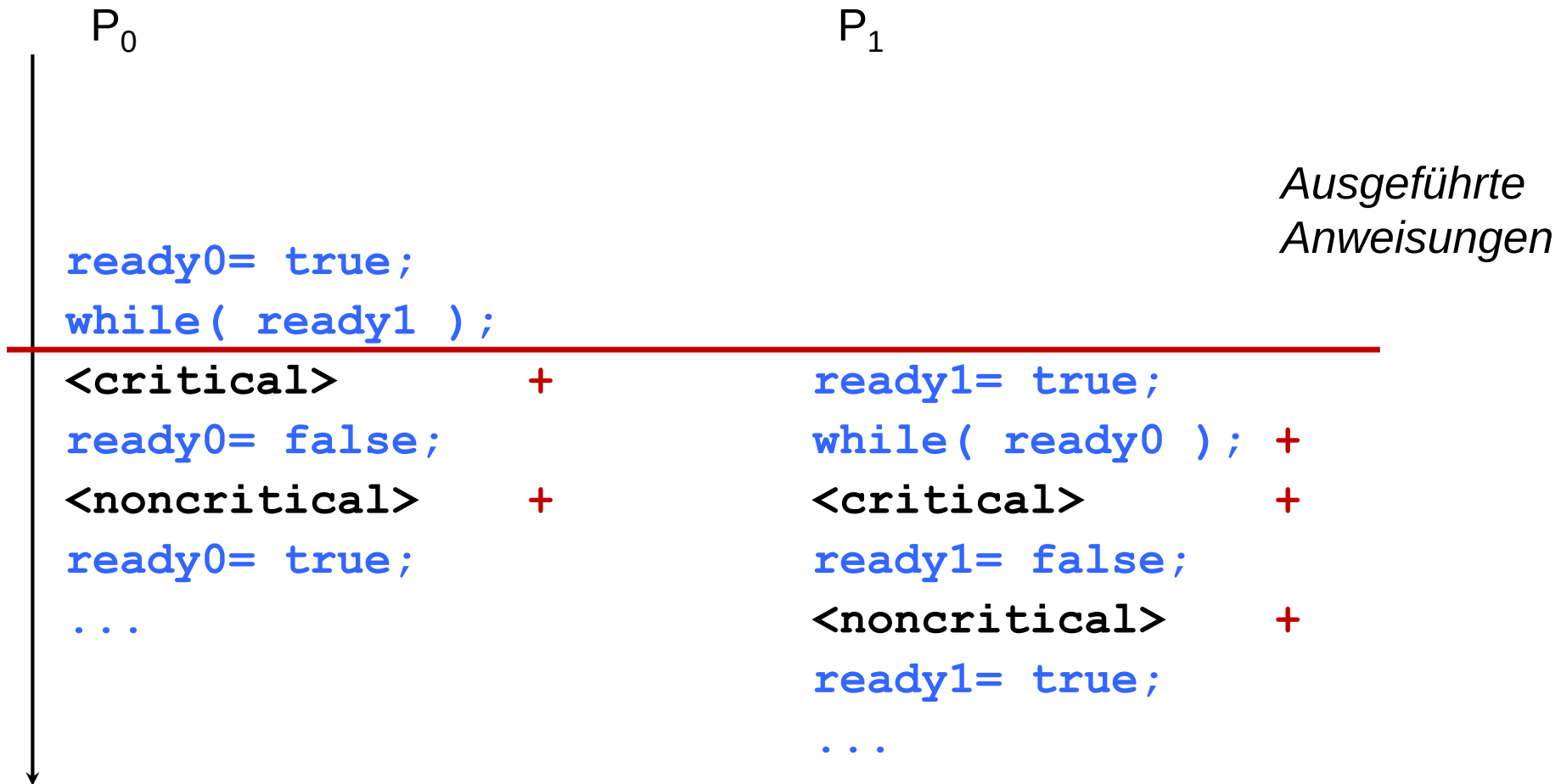
`<noncritical>` +

`ready1= true;`

`...`

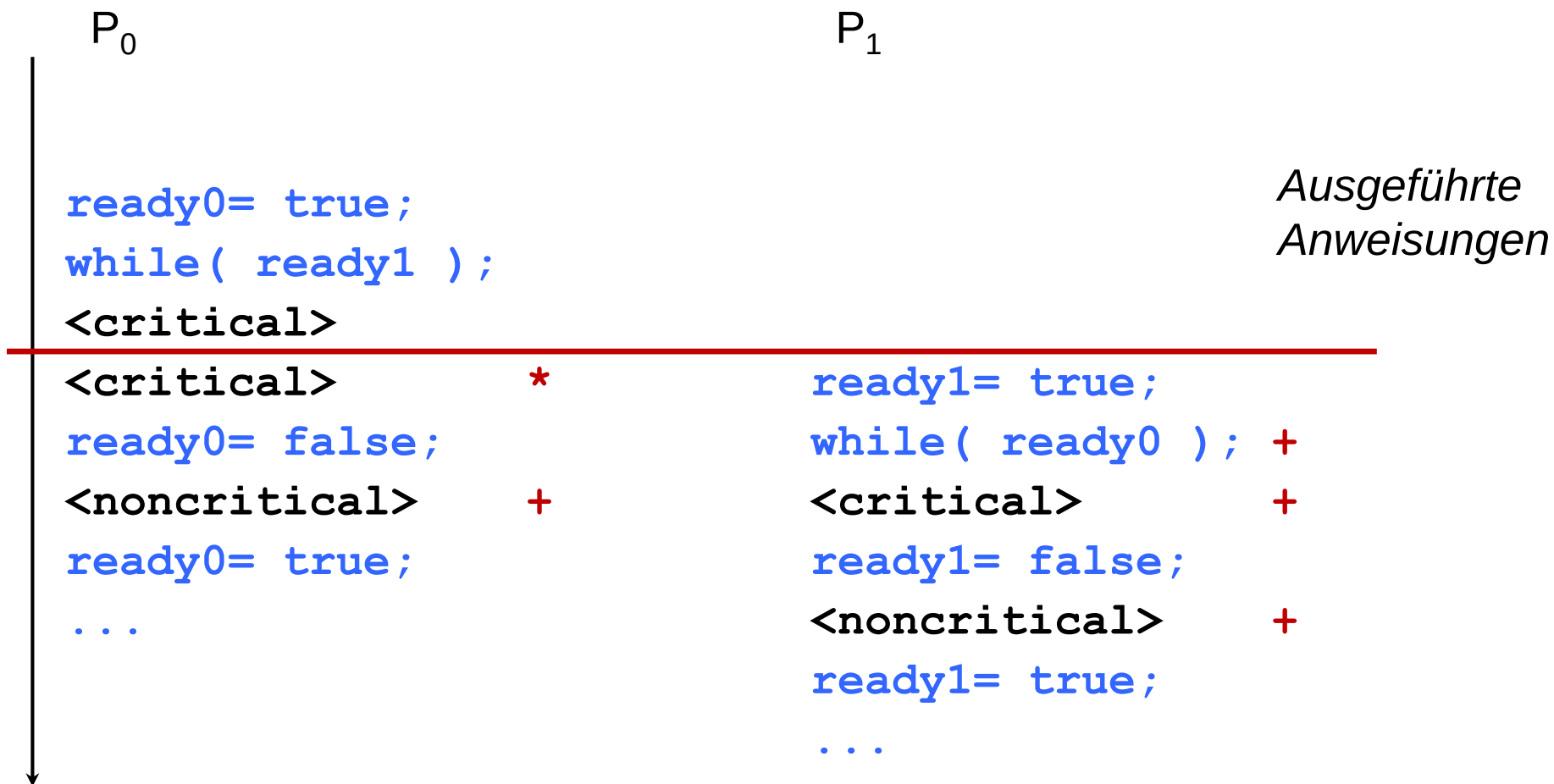
Wechselseitiger Ausschluss (7)

Harmlose Durchmischung



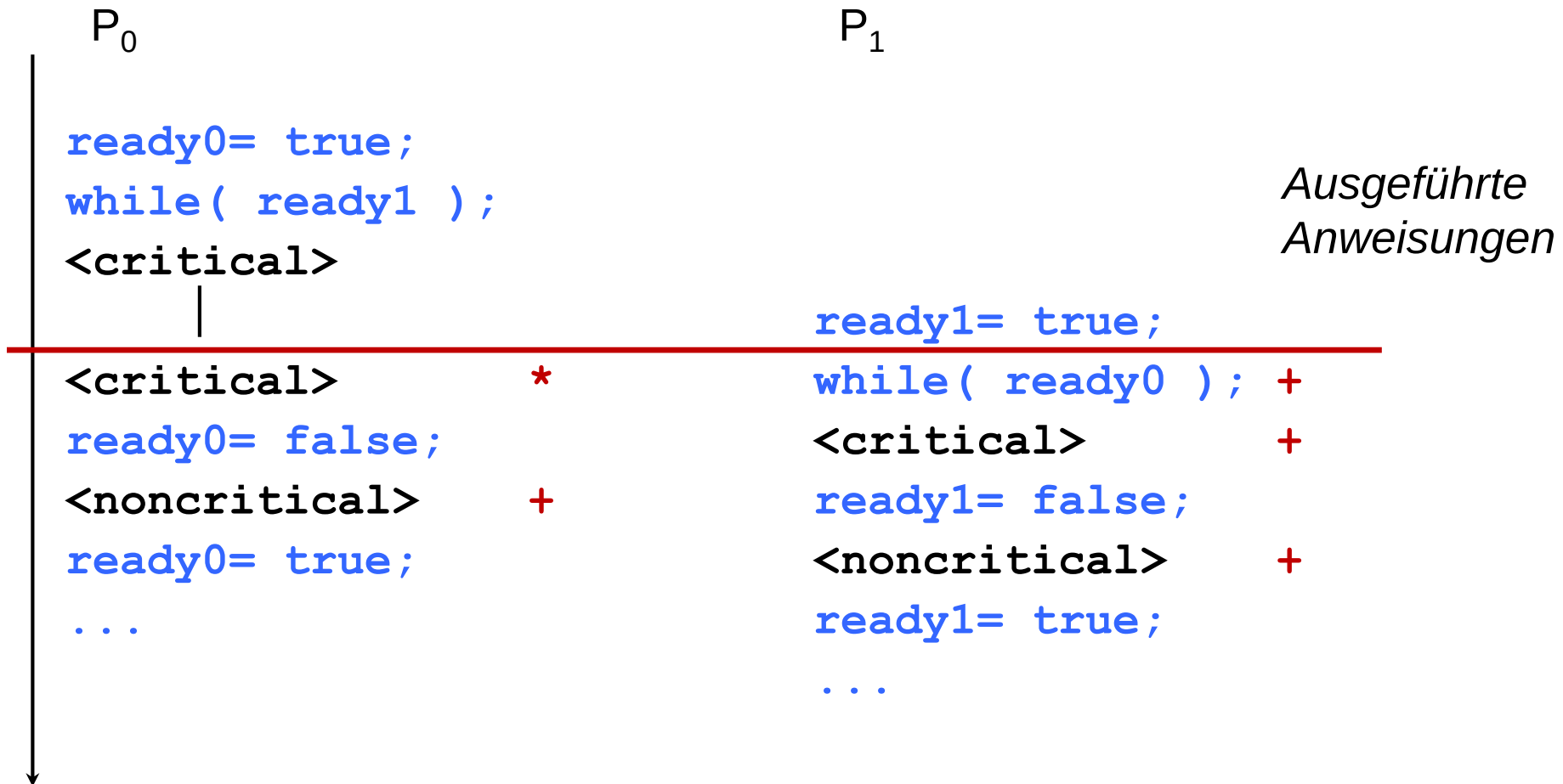
Wechselseitiger Ausschluss (7)

Harmlose Durchmischung



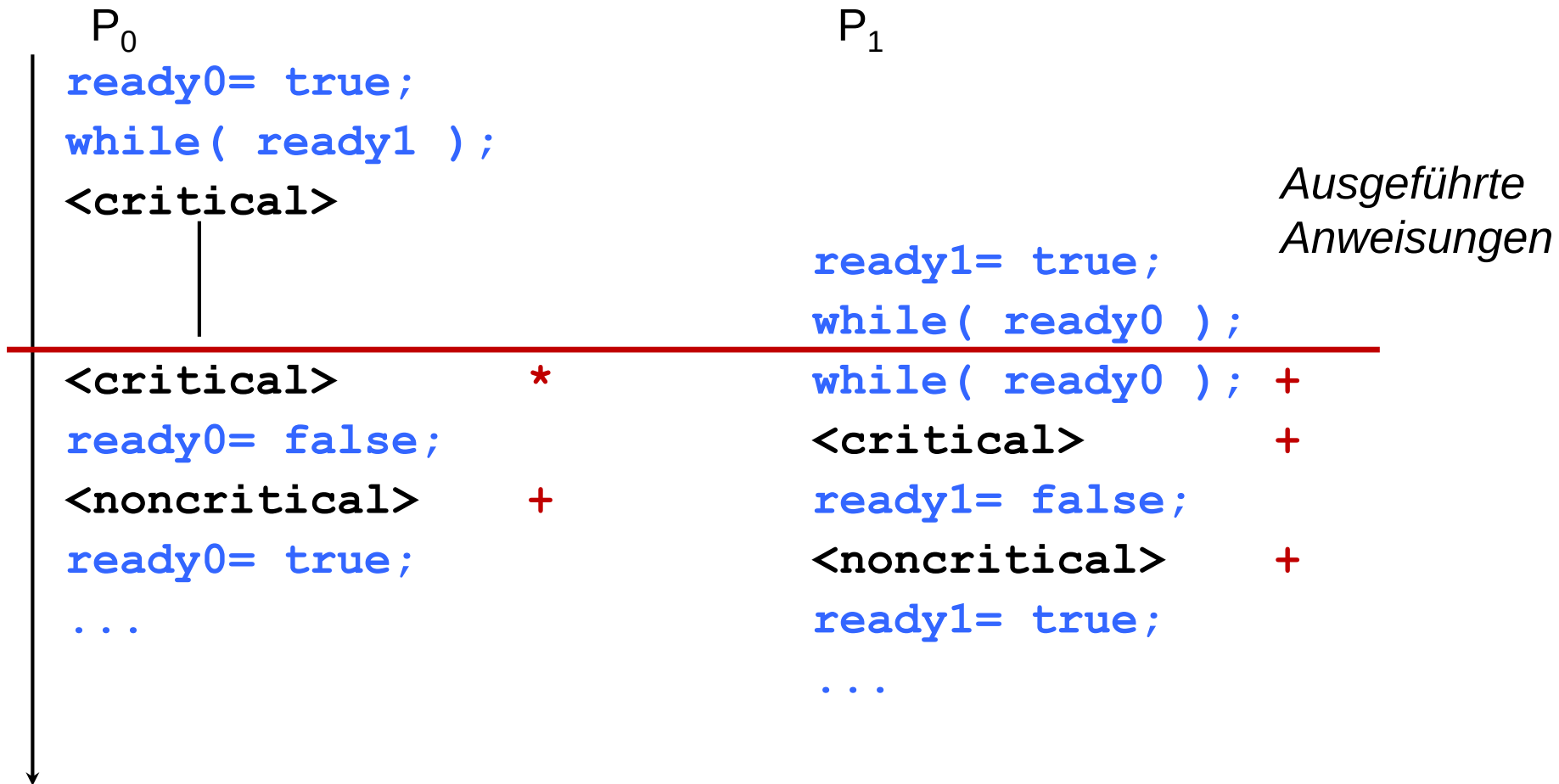
Wechselseitiger Ausschluss (7)

Harmlose Durchmischung



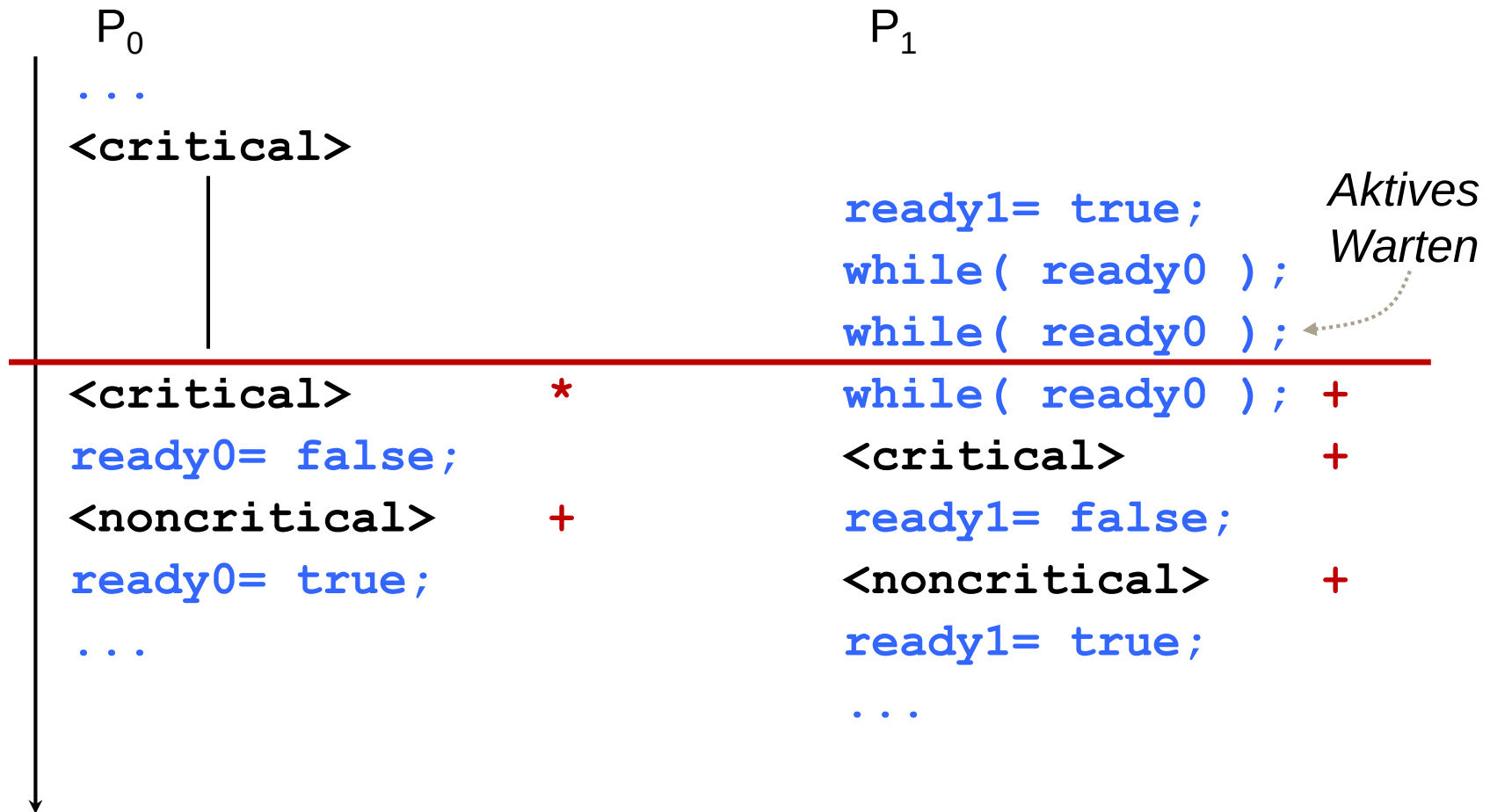
Wechselseitiger Ausschluss (7)

Harmlose Durchmischung



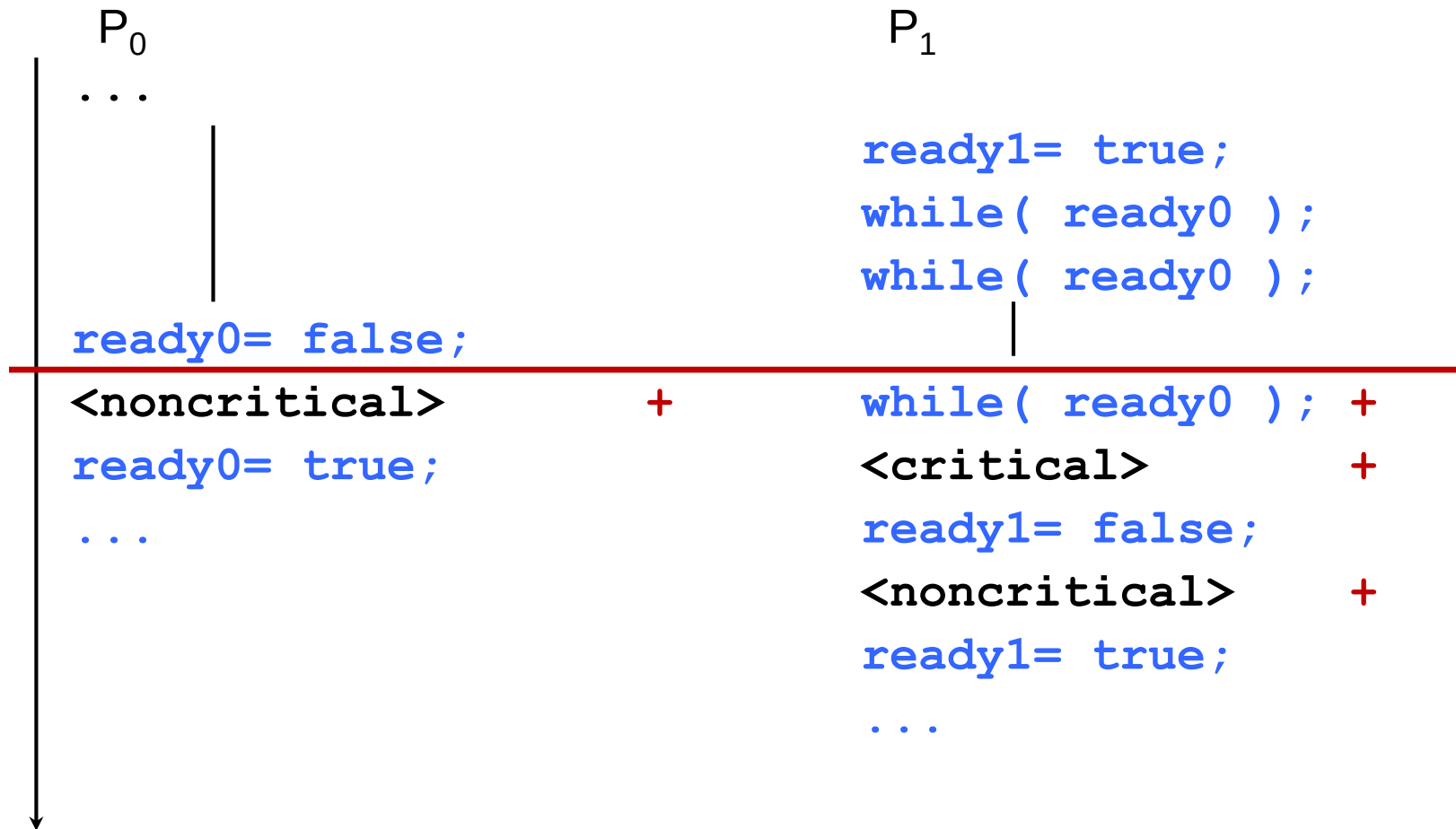
Wechselseitiger Ausschluss (7)

Harmlose Durchmischung



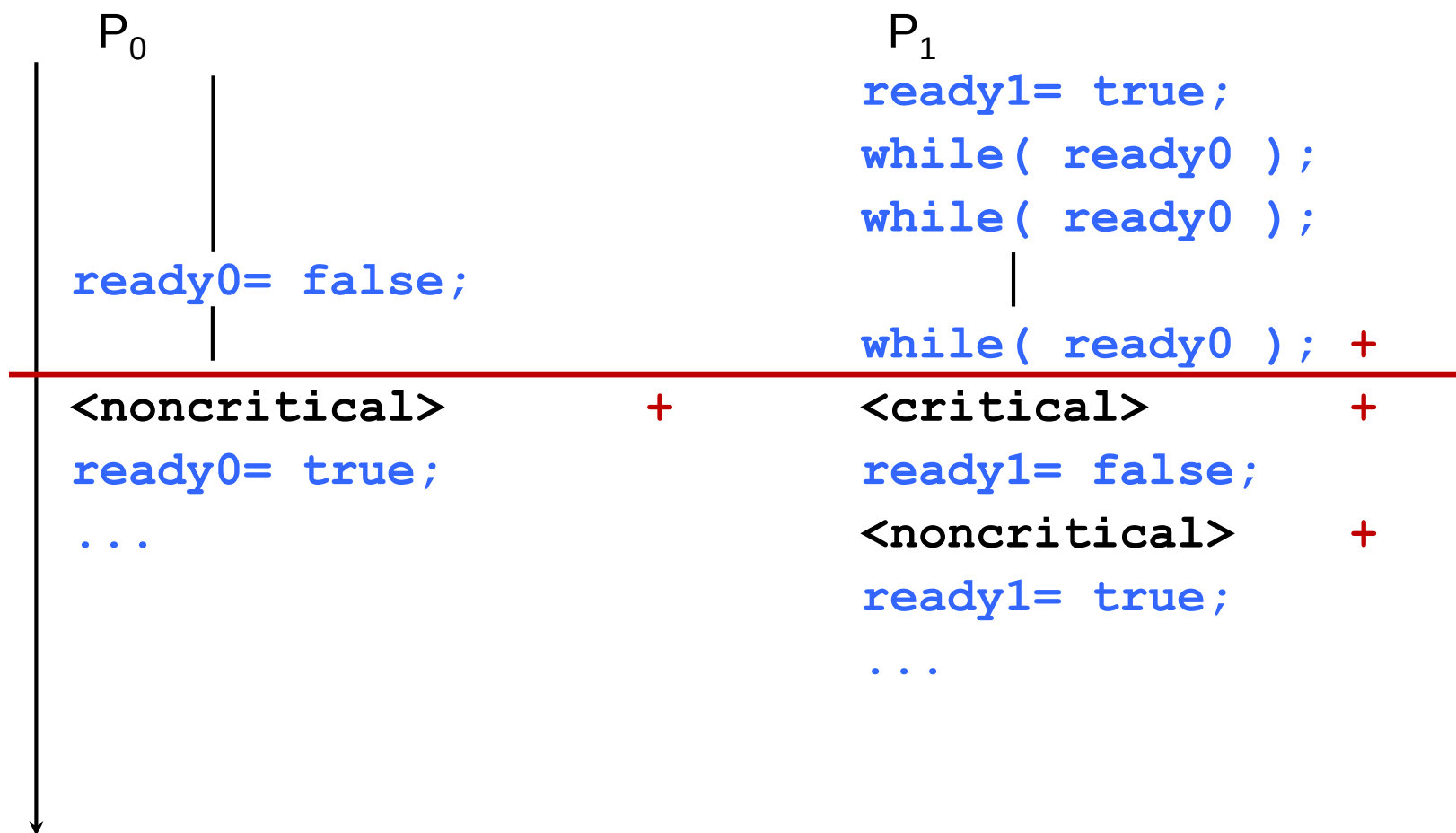
Wechselseitiger Ausschluss (7)

Harmlose Durchmischung



Wechselseitiger Ausschluss (7)

Harmlose Durchmischung



Wechselseitiger Ausschluss (8)

Deadlock

*Ausgeführte
Anweisungen*

P_0

P_1

```

ready0= true;
while( ready1 ); +
<critical>      +
ready0= false;
<noncritical>  +
ready0= true;
...

```

```

ready1= true;
while( ready0 ); +
<critical>      +
ready1= false;
<noncritical>  +
ready1= true;
...

```


Wechselseitiger Ausschluss (8)

Deadlock

*Ausgeführte
Anweisungen*

P_0

P_1

`ready0= true;`

`while( ready1 );` +

`<critical>` +

`ready0= false;`

`<noncritical>` +

`ready0= true;`

`...`

`ready1= true;`

`while( ready0 );` +

`<critical>` +

`ready1= false;`

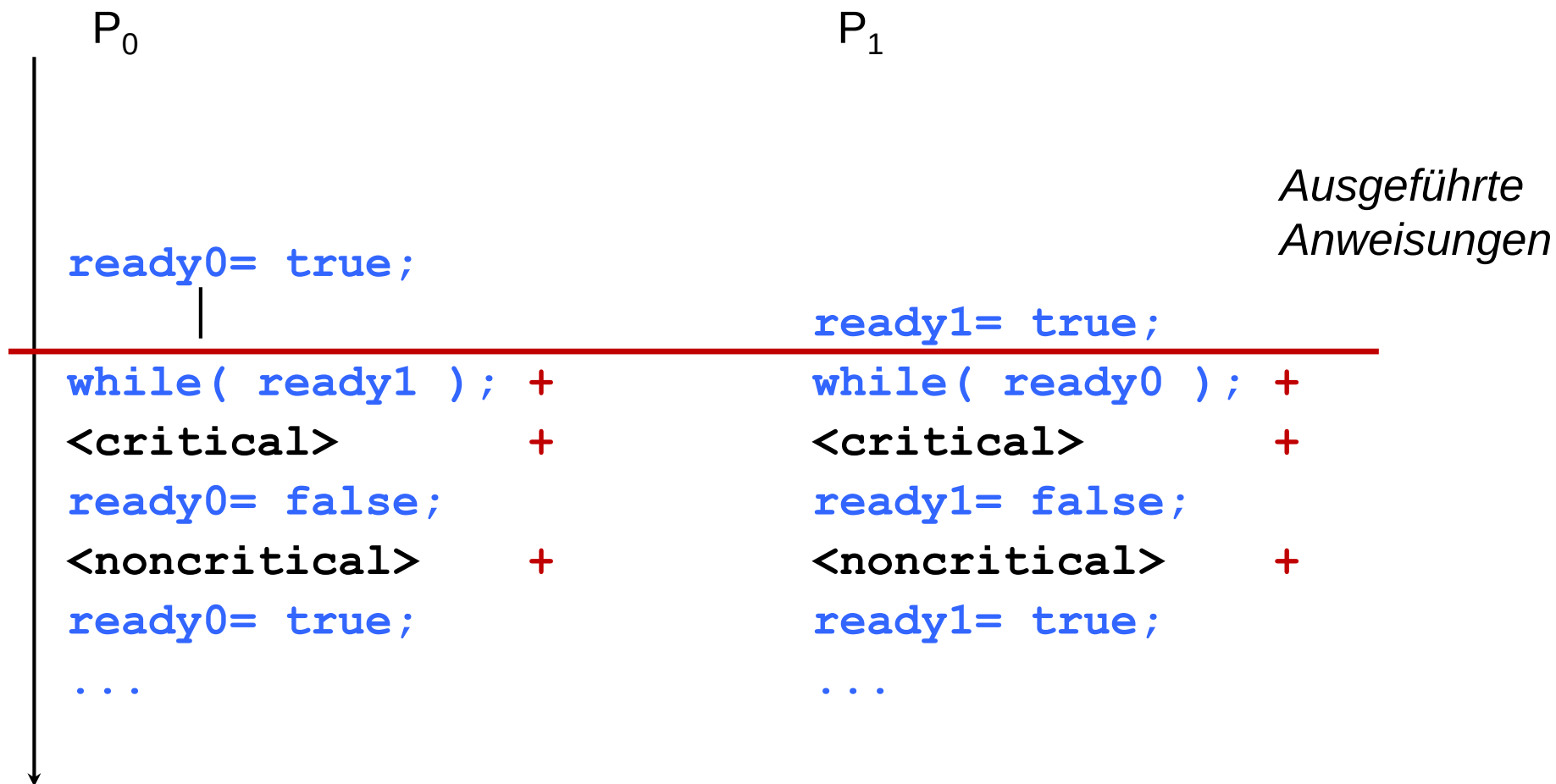
`<noncritical>` +

`ready1= true;`

`...`

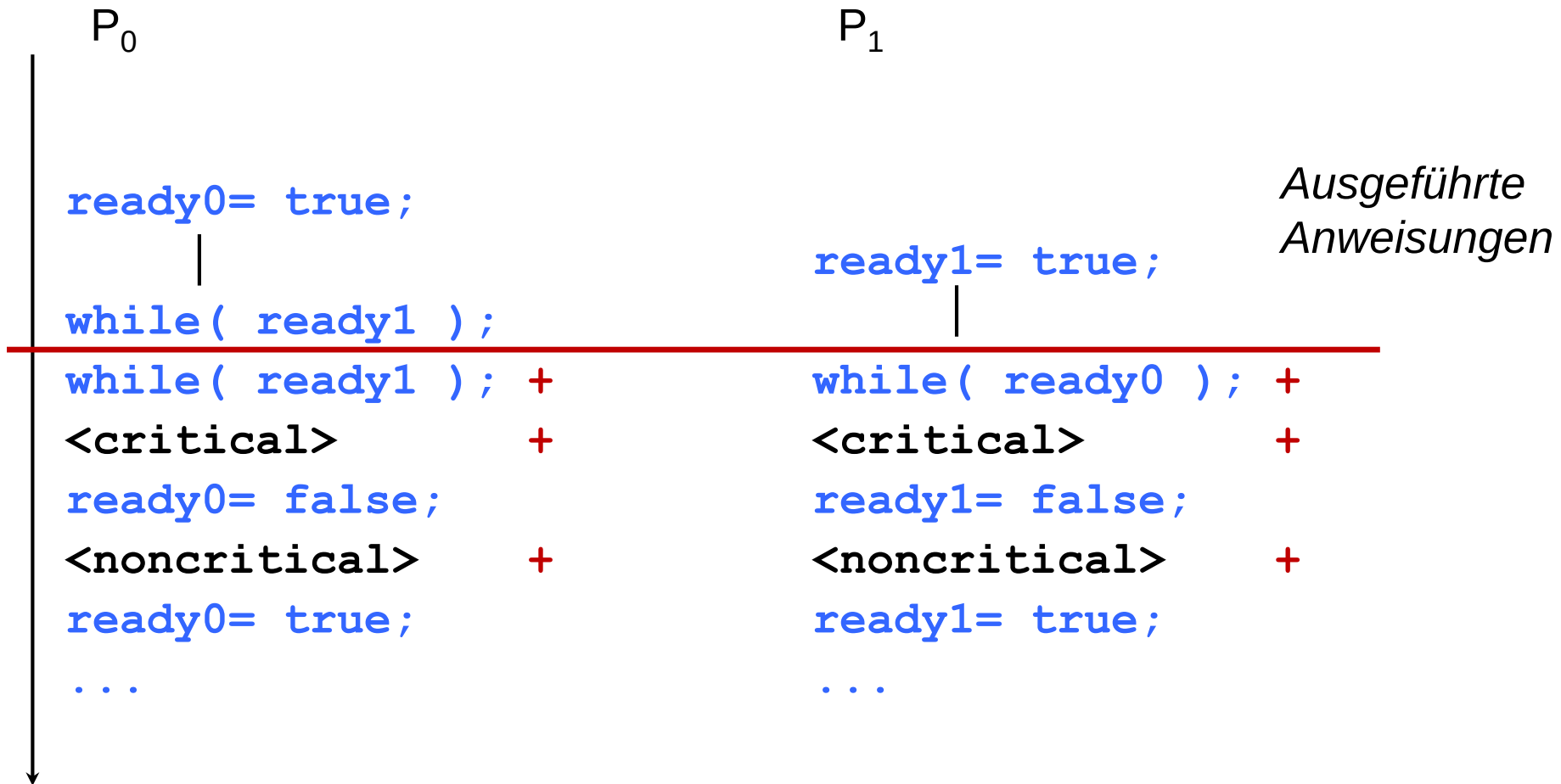
Wechselseitiger Ausschluss (8)

Deadlock



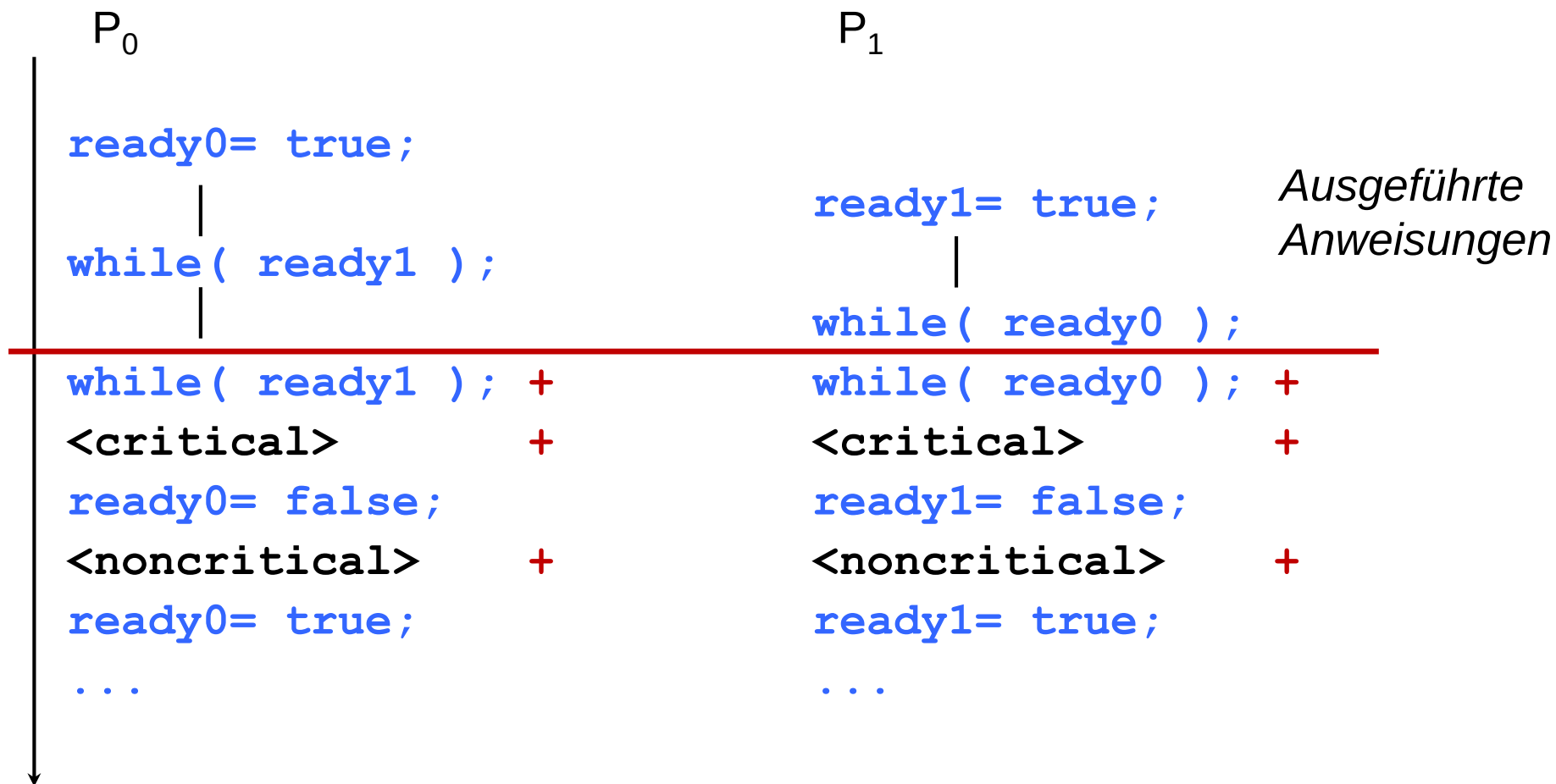
Wechselseitiger Ausschluss (8)

Deadlock



Wechselseitiger Ausschluss (8)

Deadlock



Semaphore (1)

Semaphor (griech. „Zeichenträger“)

- System-Datenstruktur mit zwei Operationen (nach Edsger W. Dijkstra)
 - P-Operation (*proberen; passeren; wait; down*)
 - Wartet bis Zugang frei
 - V-Operation (*verhogen; vrijgeven; signal; up*)
 - Macht Zugang für anderen Prozess frei
- Datenstruktur für Zugangssteuerung: *Integer*-Wert

```
class Semaphore
{
    int s;
    Semaphore( int init ) { s= init; }

    void P() { ... }
    void V() { ... }
}
```

Semaphore (3)

Implementierung kritischer Abschnitte mit einem Semaphor

```
Semaphore lock= new Semaphore( 1 );
```

Prozess 0

```
...  
while( 1 ) {  
    lock.P();  
  
    ... /* critical sec. */  
  
    lock.V();  
  
    ... /* uncritical */  
}
```

Prozess 1

```
...  
while( 1 ) {  
    lock.P();  
  
    ... /* critical sec. */  
  
    lock.V();  
  
    ... /* uncritical */  
}
```

Problem

- Konkrete Implementierung von P und V

Semaphore (6)

Vorteile einer Semaphor-Implementierung im Betriebssystem

- Einbeziehen des *Schedulers* in die Semaphor-Operationen
- Kein aktives Warten: Ausnutzen der Blockierzeit durch andere Prozesse

Implementierung einer Synchronisierung

- Zwei Prozesse P1 und P2
- Anweisung S1 in P1 soll vor Anweisung S2 in P2 stattfinden

```
Semaphore lock= new Semaphore( 0 );
```

Prozess 1

```
...  
S1;  
lock.V();  
...
```

Prozess 2

```
...  
lock.P();  
S2;  
...
```

Zusammenfassung (1)

Einordnung & Motivation

- Mehrprogramm- und Mehrbenutzerbetrieb: erlauben bessere Ausnutzung der Ressource „CPU“, falls Programme Wartezeiten haben

Prozesse

- Ein Prozess ist ein Programm, das sich in Ausführung befindet, UND dessen aktuellen Daten
- Der Prozesskontrollblock als zentrale Datenstruktur des Betriebssystems verwaltet Prozess-IDs, Speicherbereiche, Eigentümer, etc.
- Prozesse können auf einer CPU umgeschaltet werden, i.d.R. unter Kontrolle des Betriebssystems über einen *Scheduler*
- Prozesse sind in einem von 5 Zuständen (*new, ready, running, blocked/waiting, terminated*)

Zusammenfassung (3)

Aktivitätsträger (*Threads*)

- *Threads*: teilen sich Ressourcen; *Thread*-Wechsel sehr effizient
- *User-level Threads*: Anwendungsprogramm schaltet *Threads* um; keine Systemaufrufe erforderlich; blockierende Systemaufrufe blocken alle *Threads* eines Anwendungsprogramms
- *Kernel-level Threads*: Betriebssystem schaltet *Threads* um; kein Blockieren unbeteiligter *Threads*; weniger effizientes Umschalten

Parallelität und Nebenläufigkeit

- Parallelität: Anweisungen zweier Prozesse werden unabhängig voneinander zeitgleich ausgeführt; keine Parallelität auf Monoprozessor
- Nebenläufigkeit: Anweisungen können unabhängig ausgeführt werden, entweder zeitlich durchmischt oder echt zeitgleich
- Parallelität ist Spezialfall von Nebenläufigkeit

Zusammenfassung (4)

Koordinierung

- Problem: bei gemeinsamer Nutzung von Daten/Ressourcen durch mehrere Prozesse können Inkonsistenzen auftreten
- Ursache: Befehle einer Programmiersprache werden nicht atomar abgearbeitet; Prozesswechsel können innerhalb einer Anweisung oder zwischen zwei zusammengehörigen Anweisungen auftreten
- Kritischer Abschnitt: Bereich in denen Prozesse gemeinsame Ressourcen nutzen, wo aber nur ein Prozess zu einer Zeit Zugang haben soll (*mutual exclusion*)
- Algorithmus von Peterson: *mutual exclusion* per Kontroll-Variablen; aktives Warten; nur aufwändig auf mehrere/viele Prozesse erweiterbar
- Atomare Maschinenbefehle: elegant; aktives Warten
- Semaphore: Betriebssystem-Mechanismus („Zähler“) zur Steuerung von Prozessen; wartende Prozesse werden blockiert; kein aktives Warten